```
#include <iostream>
using namespace std;

int main() {
   int c, first, last, middle, n, search, array[100];

   cout << "Enter no. of elements\n";
   cin >> n;

   cout << "Enter the integer values:\n";
   for (c = 0; c < n; c++) {
      cin >> array[c];
   }

   cout << "Enter value to find\n";
   cin >> search;

   first = 0;
```

```
last = n - 1;
middle = (first + last) / 2;

while (first <= last) {
   if (array[middle] < search) {
      first = middle + 1;
   } else if (array[middle] == search) {
      cout << "Found at location \n" << middle + 1 << endl;
      break;
   } else {
      last = middle - 1;
   }
   middle = (first + last) / 2;
}

if (first > last) {
   cout << "Not found/isn't present in the list\n";
}
```

```
    return 0;
}
#include<iostream>
using namespace std;
int main()
{
        int i,n,key,a[20];
        bool status = false;
        cout<<"enter number of array elements in search key";
        cin>>n>>key;
        for(i=0;i<n;i++)
        {
                cout<<"enter elements"<<i+1;
                cin>>a[i];
                if(a[i]==key)
                cout<<"sk found at"<<i+1;
                break;
        }
        if(!status)
```

```
        cout<<"key not found";
}
bool status = false;
#include <iostream>
using namespace std;

int main() {
   int n;
   cout << "Enter number of elements: ";
   cin >> n;

   int arr[n];
   cout << "Enter elements:\n";
   for (int i = 0; i < n; i++) {
      cin >> arr[i];
   }

   // Selection sort using nested loops
   for (int i = 0; i < n; i++) {
      for (int j = i + 1; j < n; j++) {
```

```
            if (arr[i] > arr[j]) {
                // Swap immediately
                int temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
    }

    cout << "Sorted array:\n";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;

    return 0;
}
#include <iostream>
using namespace std;
```

```
int main() {
    int n;
    cout << "Enter number of elements: ";
    cin >> n;

    int arr[n];
    cout << "Enter elements:\n";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    // Insertion Sort using nested loops
    for (int i = 1; i < n; i++) {
        for (int j = i; j > 0; j--) {
            if (arr[j] < arr[j - 1]) {
                // Swap elements
                int temp = arr[j];
                arr[j] = arr[j - 1];
                arr[j - 1] = temp;
            }
```

```
      }
   }

   cout << "Sorted array:\n";
   for (int i = 0; i < n; i++) {
      cout << arr[i] << " ";
   }
   cout << endl;

   return 0;
}
#include <iostream>
using namespace std;

int main() {
   int c, first, last, middle, n, search, array[100];

   cout << "Enter no. of elements\n";
   cin >> n;
```

```
cout << "Enter the integer values:\n";
for (c = 0; c < n; c++) {
   cin >> array[c];
}

cout << "Enter value to find\n";
cin >> search;

first = 0;
last = n - 1;
middle = (first + last) / 2;

while (first <= last) {
   if (array[middle] < search) {
      first = middle + 1;
   } else if (array[middle] == search) {
      cout << "Found at location \n" << middle + 1 << endl;
      break;
   } else {
      last = middle - 1;
```

```
        }
        middle = (first + last) / 2;
    }

    if (first > last) {
        cout << "Not found/isn't present in the list\n";
    }

    return 0;
}
#include <iostream>
using namespace std;

class node {
public:
    int data;
    node *link;
};

int main() {
```

```
node *h = NULL, *cn = NULL, *t = NULL;
int val, ch;

// Creating initial linked list
do {
   cn = new node();
   cout << "Enter node data: ";
   cin >> val;
   cn->data = val;
   cn->link = NULL;

   if (h == NULL) {
      h = cn;
      t = cn;
   } else {
      t->link = cn;
      t = cn;
   }
   cout << "\nDo you want to continue 1/0:\n";
   cin >> ch;
```

```
} while (ch);

// Display original linked list data
cout << "\nLinked list data:\n";
t = h;
while (t != NULL) {
   cout << t->data << " -> ";
   t = t->link;
}
cout << "NULL\n";

// Deletion at starting
if (h != NULL) {
   t = h;
   h = h->link;
   t->link = NULL;
   delete t;
   cout << "\nDeletion completed at beginning.\n";
} else {
   cout << "\nList is empty, deletion not possible.\n";
```

```
    }

    // Display linked list data after deletion
    cout << "\nLinked list data after deletion operation:\n";
    t = h;
    while (t != NULL) {
        cout << t->data << " -> ";
        t = t->link;
    }
    cout << "NULL\n";

    return 0;
}
#include <iostream>
using namespace std;

int main() {
    int s[10], size, item, top = -1, op;

    cout << "Enter size of the stack: ";
```

```
    cin >> size;


    do {
        cout << "\n1: PUSH\n2: POP\n3: PEEK\n4: DISPLAY\n5:
EXIT\nCHOOSE ANY ONE OPTION: ";
        cin >> op;


        switch (op) {
            case 1:
                if (top == size - 1) {
                    cout << "Stack is overflow\n";
                } else {
                    top = top + 1;
                    cout << "\nEnter stack elements: ";
                    cin >> item;
                    s[top] = item;
                }
                break;
            case 2:
                if (top == -1) {
```

```
        cout << "Stack is underflow\n";
    } else {
        cout << s[top] << " is popped\n";
        top = top - 1;
    }
    break;
case 3:
    if (top == -1) {
        cout << "Stack is underflow\n";
    } else {
        cout << s[top] << " is top element\n";
    }
    break;
case 4:
    if (top == -1) {
        cout << "Stack is underflow\n";
    } else {
        cout << "Stack Elements:\n";
        for (int i = top; i >= 0; i--) {
            cout << s[i] << " ";
```

```
                }
                cout << "\n";
            }
            break;
        case 5:
            exit(0);
        default:
            cout << "Invalid option\n";
      }
      cout << "Do you want to continue 1/0? ";
      cin >> op;
   } while (op);

   return 0;
}
#include <iostream>
using namespace std;

class node {
public:
```

```
    int data;
    node *link;
};

int main() {
    node *top = NULL, *cn = NULL, *t = NULL;
    int val, op;

    do {
        cout << "\n1: PUSH\n2: POP\n3: PEEK\n4: DISPLAY\n5:
EXIT\nCHOOSE ANY ONE OPTION: ";
        cin >> op;

        switch (op) {
            case 1:
                cn = new node();
                cout << "\nEnter node data: ";
                cin >> val;
                cn->data = val;
                cn->link = NULL;
```

```
    if (top == NULL) {
        top = cn;
        t = cn;
    } else {
        cn->link = top;
        top = cn;
    }
    break;

case 2:
    if (top == NULL) {
        cout << "Stack is underflow\n";
    } else {
        cout << top->data << " is popped\n";
        t = top;
        top = top->link;
        t->link = NULL;
        delete t;
    }
```

```
    break;

case 3:
    if (top == NULL) {
        cout << "Stack is underflow\n";
    } else {
        cout << top->data << " is top node value\n";
    }
    break;

case 4:
    if (top == NULL) {
        cout << "Stack is underflow\n";
    } else {
        t = top;
        while (t != NULL) {
            cout << t->data << " ";
            t = t->link;
        }
        cout << "\n";
```

```
            }
            break;

        case 5:
            exit(0);

        default:
            cout << "Invalid option\n";
      }
      cout << "Do you want to continue 1/0? ";
      cin >> op;
   } while (op);

   return 0;
}
```

Queue using arrays

```
#include <iostream>
using namespace std;

int main() {
```

```
    int q[100], item, f = -1, r = -1, size, choice, cont;


    cout << "\nEnter size of Queue: ";
    cin >> size;


    do {
        cout << "\n1: Enqueue\n2: Dequeue\n3: Peek\n4: Display\n5:
Exit\n";
        cout << "\nChoose any one option: ";
        cin >> choice;


        switch (choice) {
            case 1:
                if (r == size - 1) {
                    cout << "\nQueue Overflow\n";
                } else {
                    if (f == -1) {
                        f = 0;
                    }
                    cout << "Enter item: ";
```

```
        cin >> item;
        r++;
        q[r] = item;
    }
    break;

case 2:
    if (f == -1 || f > r) {
        cout << "\nQueue Underflow\n";
    } else {
        cout << "\nDeleted item: " << q[f] << endl;
        f++;
        if (f > r) { // Reset queue configuration if empty
            f = r = -1;
        }
    }
    break;

case 3:
    if (f == -1 || f > r) {
```

```
        cout << "\nQueue Underflow/Empty\n";
    } else {
        cout << "\nFront element: " << q[f] << endl;
    }
    break;

case 4:
    if (f == -1 || f > r) {
        cout << "\nQueue Underflow\n";
    } else {
        cout << "\nQueue elements: ";
        for (int i = f; i <= r; i++) {
            cout << q[i] << " ";
        }
        cout << "\n";
    }
    break;

case 5:
    exit(0);
```

```
            default:
                cout << "\nInvalid option\n";
        }

        cout << "\nEnter 1 to continue OR 0 to exit: ";
        cin >> cont;
    } while (cont);

    return 0;
}
```

Queue using linked list

```
#include <iostream>
using namespace std;

class node {
public:
    int data;
    node *link;
};
```

```
int main() {
    node *f = NULL, *r = NULL, *cn = NULL, *t = NULL;
    int ch;

    do {
        cout << "\n1: Enqueue\n2: Dequeue\n3: Peek\n4: Display\n5: Exit\n";
        cout << "\nEnter your choice: ";
        cin >> ch;

        switch (ch) {
            case 1: // Enqueue
                cn = new node();
                cout << "Enter item: ";
                cin >> cn->data;
                cn->link = NULL;

                if (f == NULL) {
                    f = r = cn;
```

```
    } else {
        r->link = cn;
        r = cn;
    }
    cout << cn->data << " is inserted into Queue\n";
    break;

case 2: // Dequeue
    if (f == NULL) {
        cout << "\nQueue Underflow\n";
    } else {
        t = f;
        cout << "\nDeleted item: " << t->data << endl;
        f = f->link;
        delete t;
        if (f == NULL) {
            r = NULL;
        }
    }
    break;
```

```
case 3: // Peek
    if (f == NULL) {
        cout << "\nQueue is empty\n";
    } else {
        cout << "\nFront element: " << f->data << endl;
    }
    break;

case 4: // Display
    if (f == NULL) {
        cout << "\nQueue is empty\n";
    } else {
        t = f;
        cout << "\nQueue elements: ";
        while (t != NULL) {
            cout << t->data << " ";
            t = t->link;
        }
        cout << "\n";
```

```
            }
            break;

        case 5:
            cout << "\nExiting...\n";
            exit(0);

        default:
            cout << "\nInvalid choice\n";
      }

      cout << "\nEnter 1 to continue 0 to Exit: ";
      cin >> ch;
   } while (ch);

   return 0;
}
```